Fundamentos de Contenedores

Unidad 1

Evolución de Infraestructura • Docker • Ciclo de Vida • Laboratorio Práctico

Curso de Contenedores y Orquestación

Lucas Zuluaga P.
Jonathan Betancur A.

Agenda

Contenido de la Unidad 1

Evolución de la Infraestructura
De Bare Metal a Contenedores
01

¿Qué es un Contenedor?
Concepto, aislamiento y portabilidad
02

Componentes de Docker
Engine, imágenes, volúmenes, redes
03

Ciclo de Vida del Contenedor
Created → Running → Stopped → Removed
04

Laboratorio Práctico
3 contenedores: Nginx, PostgreSQL, Flask
05

Evolución de la Infraestructura

Del servidor físico al contenedor ligero

Bare Metal

1990s

Un SO por servidor.
Recursos desperdiciados.
Escalado costoso.

Máquinas Virtuales

2000s

Múltiples SO virtualizados.
Hypervisor gestiona HW.
Mejor uso de recursos.

Contenedores

2013+

Procesos aislados.
Compartiendo kernel.
Arranque en segundos.

Bare Metal

Servidor físico dedicado — una aplicación por máquina

Características

- Hardware dedicado sin capa de virtualización
- Un único Sistema Operativo por servidor
- Máximo rendimiento posible (sin overhead)
- Escalado vertical: más RAM, CPU, disco
- Aprovisionamiento lento (días/semanas)
- Alto desperdicio de recursos en baja carga

Arquitectura

Máquinas Virtuales

Virtualización completa del hardware mediante hypervisor

Características

- Hypervisor abstrae el hardware físico
- Cada VM tiene su propio SO completo
- Aislamiento fuerte entre máquinas virtuales
- Mayor aprovechamiento que Bare Metal
- Overhead significativo (GB de RAM por VM)
- Arranque en minutos, no en segundos

Arquitectura

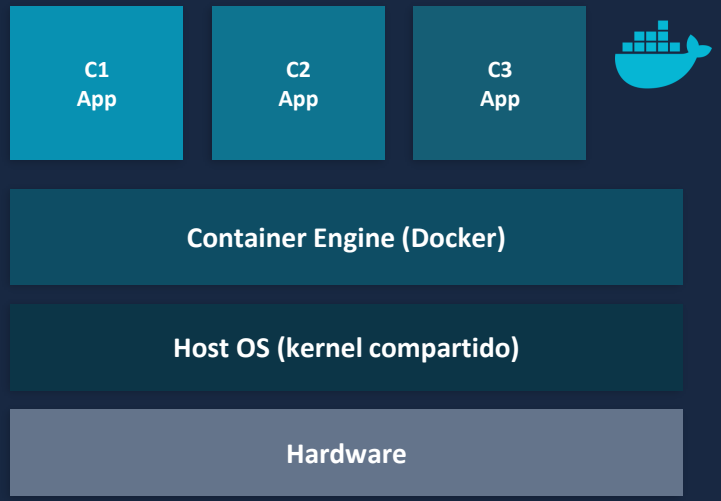
Contenedores

Virtualización a nivel de sistema operativo — aislamiento de procesos

Características

- Comparten el kernel del host OS
- Aislamiento mediante namespaces y cgroups (control groups)
- Arranque en milisegundos, no minutos
- Imagen de ~MB vs GB de una VM
- Portabilidad total: dev = staging = prod
- Densidad alta: cientos por servidor

Arquitectura



Comparativa

Bare Metal vs Máquinas Virtuales vs Contenedores

Característica	Bare Metal	Máquina Virtual	Contenedor
Arranque	Minutos	Minutos	Segundos/ms
Tamaño	N/A (físico)	GBs	MBs
Aislamiento	Físico total	SO completo	Namespaces/cgroups
Densidad	1 app/server	~10-20 VMs	~100s contenedores
Portabilidad	Nula	Media (OVA)	Total (imagen)
Overhead	Ninguno	Alto (hypervisor)	Mínimo
Escalado	Vertical	Vertical/Horizontal	Horizontal nativo

¿Qué es un Contenedor?

Unidad estándar de software que empaqueta código y dependencias

Un contenedor es una instancia ejecutable de una imagen que encapsula una aplicación junto con todas sus dependencias, librerías y configuración, ejecutándose de forma aislada sobre el kernel del sistema operativo host.

Namespaces

Aíslan PID, red, filesystem, usuarios e IPC entre contenedores

Cgroups

Limitan y controlan CPU, memoria, I/O y recursos del sistema

Union FS

Sistema de archivos por capas (overlay2) para imágenes eficientes

OCI Standard

Open Container Initiative define el estándar de runtime e imágenes

Características de un Contenedor

Beneficios técnicos clave de la contenedorización

Ligero

Comparte el kernel del host, sin SO completo. Tamaño en MB.

Portable

Se ejecuta igual en dev, staging y producción sin cambios.

Aislado

Cada contenedor tiene su propio filesystem, red y procesos.

Inmutable

Las imágenes son de solo lectura. Cambios en capa superior.

Escalable

Se replica horizontalmente con orquestadores como Kubernetes.

Versionado

Cada imagen tiene tags. Rollback instantáneo ante fallos.

Componentes de Docker

Ecosistema completo para construir, distribuir y ejecutar contenedores

Docker Engine

Daemon (dockerd) + CLI + API REST. El corazón de Docker.

Docker Images

Plantillas de solo lectura con capas. Base de los contenedores.

Containers

Instancias ejecutables de imágenes con capa de escritura.

Dockerfile

Archivo de instrucciones para construir imágenes paso a paso.

Docker Hub

Registry público con miles de imágenes oficiales y comunitarias.

Volumes

Persistencia de datos independiente del ciclo de vida del contenedor.

Networks

Comunicación aislada entre contenedores (bridge, host, overlay).

Compose

Define aplicaciones multi-contenedor con un solo archivo YAML.

Docker Engine

Arquitectura cliente-servidor del motor de contenedores

Docker CLI

```
docker build
docker run
docker pull
docker push
docker ps
```

REST API

Docker Daemon

dockerd gestiona:

- Imágenes
- Contenedores
- Redes
- Volúmenes

Runtime

containerd

+

runc

(OCI spec)

Dockerfile e Imágenes

De instrucciones a plantillas ejecutables

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Capas de la Imagen

Capa 7: CMD (metadata)

Capa 6: EXPOSE 5000

Capa 5: COPY . .

Capa 4: RUN pip install

Capa 3: COPY requirements.txt

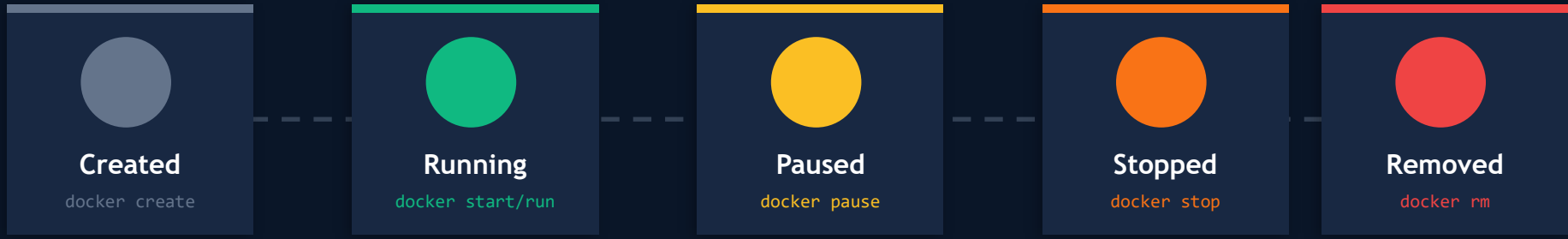
Capa 2: WORKDIR /app

Capa 1: python:3.11-slim (base)

↑ Read-only layers (inmutables)

Ciclo de Vida del Contenedor

Estados y transiciones desde la creación hasta la eliminación



`docker create <img>`	Crea el contenedor sin ejecutarlo. Sistema de archivos listo.
`docker run <img>`	Crea + inicia el contenedor. Combina create + start.
`docker pause / unpause`	Congela/descongela procesos usando cgroups freezer.
`docker stop <id>`	Envía SIGTERM, luego SIGKILL tras timeout (10s por defecto).
`docker rm <id>`	Elimina el contenedor y su capa de escritura. Datos perdidos sin volumen.

Laboratorio Práctico

Instalación de Docker + 3 Contenedores en Linux

01

Nginx — Servidor Web

02

PostgreSQL — Base de Datos

03

Python Flask — Aplicación Web

Lab 1: Nginx — Servidor Web

Contenedor con servidor web y página personalizada

Dockerfile

```
FROM nginx:alpine

# Copiar página HTML personalizada
COPY index.html /usr/share/nginx/html/

# Exponer el puerto 80
EXPOSE 80

# Nginx ya tiene CMD por defecto
CMD ["nginx", "-g", "daemon off;"]
```

Despliegue paso a paso

```
# Crear directorio del proyecto
mkdir -p ~/lab1-nginx && cd ~/lab1-nginx

# Crear index.html
echo 'Lab 1: Nginx en Docker' \
  > index.html

# Construir la imagen
docker build -t mi-nginx:v1 .

# Ejecutar el contenedor
docker run -d --name web-nginx \
  -p 8080:80 mi-nginx:v1

# Verificar
curl http://localhost:8080
docker logs web-nginx
```

Lab 2: PostgreSQL — Base de Datos

Contenedor con persistencia de datos usando volúmenes

Dockerfile

```
FROM postgres:16-alpine

# Variables de entorno para configuración
ENV POSTGRES_DB=labdb
ENV POSTGRES_USER=labuser
ENV POSTGRES_PASSWORD=labpass123

# Script de inicialización
COPY init.sql /docker-entrypoint-initdb.d/

EXPOSE 5432
```

Despliegue paso a paso

```
# Crear directorio del proyecto
mkdir -p ~/lab2-postgres && cd ~/lab2-postgres

# Crear script de inicio SQL
cat > init.sql << 'EOF'
CREATE TABLE alumnos (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100),
  curso VARCHAR(50)
);
INSERT INTO alumnos VALUES
  (1,'Ana','Containers');
EOF

# Construir y ejecutar con volumen
docker build -t mi-postgres:v1 .
docker run -d --name db-postgres \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data \
  mi-postgres:v1
```

Lab 3: Python Flask — App Web

Aplicación web personalizada con Dockerfile multi-instrucción

Dockerfile

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Instalar dependencias primero (cache)
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY app.py .

EXPOSE 5000
CMD ["python", "app.py"]
```

Despliegue paso a paso

```bash
# Crear directorio del proyecto
mkdir -p ~/lab3-flask && cd ~/lab3-flask

# Crear requirements.txt
echo 'flask==3.0.0' > requirements.txt

# Crear app.py
cat > app.py << 'EOF'
from flask import Flask
import os, socket
app = Flask(__name__)
@app.route('/')
def hello():
  host = socket.gethostname()
  return f'Lab3 Flask - Host: {host}'
app.run(host='0.0.0.0', port=5000)
EOF

# Construir y ejecutar
docker build -t mi-flask:v1 .
docker run -d --name app-flask \
  -p 5000:5000 mi-flask:v1
```

Comandos Útiles de Docker

Referencia rápida para gestión de contenedores

Gestión de Contenedores

`docker ps`

Listar contenedores activos

`docker ps -a`

Listar todos (incluidos detenidos)

`docker stop <name>`

Detener un contenedor

`docker rm <name>`

Eliminar un contenedor

`docker logs -f <name>`

Ver logs en tiempo real

`docker exec -it <name> bash`

Abrir shell dentro del contenedor

Gestión de Imágenes

`docker images`

Listar imágenes locales

`docker rmi <img>`

Eliminar una imagen

`docker build -t name:tag .`

Construir imagen desde Dockerfile

`docker pull <img>`

Descargar imagen de registry

`docker tag <img> new:tag`

Etiquetar una imagen

`docker system prune -a`

Limpiar todo lo no utilizado

Resumen — Unidad 1

- La infraestructura evolucionó de Bare Metal → VMs → Contenedores
- Los contenedores aíslan procesos compartiendo el kernel del host
- Docker provee el ecosistema completo: Engine, Images, Volumes, Networks
- El ciclo de vida: Created → Running → Paused → Stopped → Removed
- Desplegamos 3 contenedores: Nginx, PostgreSQL y Flask